MODEL- notes

viernes, 23 de enero de 2026 13:22

FINAL OBJECTIVE :
Generic two-spool power-balance dynamics + simplified 0-D cycle + plausible maps + full test harness

Stage 0 (v.0.0)-
Repo Structure :

```
FADEC-SIM/
 ├– README.md
 ├– .gitignore
 ├– docs/
 |        ├– images/
 |        ├– references/
 |        └– OneNote
 ├ scripts/
 |        └– main.m
 |        └– run_simulations.m
 |        └– plots.m
 |        └– init_params.m
 ├ model/
 |        ├– fadec_sim.slx
 |        ├– subsystems/
 |           └– .gitkeep
 ├ results/
 |              ├– plots/
 |                 └– .gitkeep
 |              ├– metrics/
 |                 └– .gitkeep
 |              ├– logs/
 |                 └– .gitkeep
 ├ report/
 └– .gitkeep
```

Stage 1 (v0.1) -

$$\text{Input} \xrightarrow{\text{Map}} N_ref \xrightarrow{\text{PI}} Wf_raw \xrightarrow{\text{Limits}} Wf_cmd \xrightarrow{\text{Engine}} \text{Outputs}$$

- **Goal:** build a minimal, believable FADEC-style closed loop that converts a normalized throttle command into a fuel command, drives a simple engine spool-speed model, and produces proxy outputs (EGT and thrust) for visualization and testing

- **Input:** Normalized Throttle command (0–1).

Can be a step block, or throttle curve. Clean signal for stage 1.
Engines dont start from 0, they idle so we need to define an initial engine state, important for realism and stability.

N(t = 0) = N_iddle + throttle(0) ∗(N_maxref − N_idle)

- **Controller block:** Converts throttle into a engine spool speed reference and uses a simple feedback controller to compute a raw fuel command .

  This is done by computing an error *e_N = N_ref - N* together with a PI control.

$$Wf_{raw} = Wf_P + Wf_I + Wf_{ff} = Kp_N \cdot e_N + \int Ki_N \cdot e_N dt + \frac{N_{ref} - b_{Neq}}{K_{Neq}}$$

Include basic actuator realism . This means fuel delivery cannot go below zero, exceed a maximum or change infinitely fast.

<u>Saturation</u> - clamps fuel command to stay in a min/max.

sat(Wf_raw) → [0,1]

<u>Rate Limiting</u> - Fuel system can only open/close valves so fast

Wf_cmd is *Wf_raw* after these realism constraints

- **Engine (plant) block:** A minimal dynamic model that turns fuel command into normalized spool speed .

  This is achieved by a first-order lag model, where the time constant tau_N is the engine responsiveness. This models the inertia of the spool.

$$\frac{dN}{dt} = \frac{N_{eq}(Wf_{cmd}) - N}{\tau_N}$$

We are not modelling torque yet so we can map fuel command to an equilibrium steady-state spool speed. Later stages will implement a more complex f().

We also obtain two derived outputs: an EGT and thrust proxy.

The <u>Exhaust Gas Temperature proxy</u> is placeholder output signal.
We can use the following "trend-correct" equation

$$EGT(t) = EGT_{idle} + A \cdot Wf_{cmd}(t) - B \cdot N(t)$$

Added a lag to EGT to behave like real temperature measurement

The <u>Thrust proxy</u> can be modelled with a power law.

$$Thrust(t) = T_{max} \cdot N(t)^a$$

This is not claiming real thermodynamics or physics , the proxies are controllable believable signals for stage 1. Later stages will use more realistic calculations.

- **Outputs:**
  - Wf_cmd
  - N
  - EGT_proxy
  - Thrust_proxy

Limitations:

- Plant is 1st order + linear map
- Proxies are trend-only
- No EGT limiter / overspeed protection yet

Stage 2 (v0.2) -

Ideas:

- Upgrade input signal generator
- EGT limiter
- Control Tunning
- Overspeed limiter (N_max protection)
- Proper anti-windup (back-calc)
- N_ref ramp/accel schedule (command shaping)
- EGT heat/cool dynamics (2 time constants)
- Sensor lag + noise (N, EGT)
- Nonlinear fuel→N_eq map (lookup table)
- Simple fault hooks (stuck fuel valve / sensor bias)

Upgrade Throttle Input

Replace the current “single Step throttle” with a flexible throttle input generator so we can run more realistic and more stressful tests (ramps, burst–chop, curves) all from main.

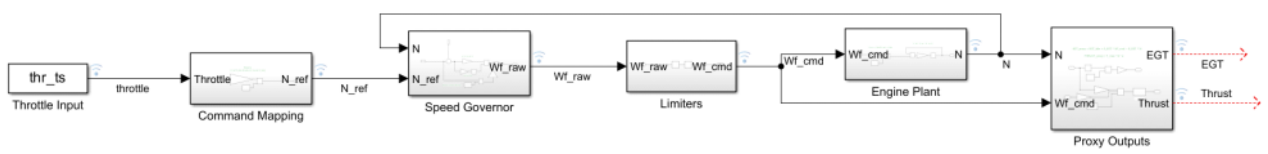
We currently have a Step signal thr(t) ∈ [0,1] defined by three variables
(thr_init , thr_final , thr_step_time)

We will generate the throttle as a timeseries in MATLAB, and feed it into Simulink using a From Workspace block. The Simulink model will always read a single signal called *thr_ts* and nothing else.
This makes the model independent from the test type: the model does not care if the input was a step, ramp, or burst–chop; it just receives the time history.

In main, we define a test case with a profile field ("step", "ramp", "burstChop", "custom"). Based on this, we build thr_ts and also set stopTime consistently.

This proyect is going to focus on BurstChop following NASA's paper.

Subsystems



Organised the model into a controller–limiter–plant pipeline that matches typical NASA's engine-control layouts.

EGT Limiter

Right now the Limiter subsystem is basically taking one input, *Wf_raw*,then applying the saturation limits and the rate limiter, and outputs *Wf_cmd*.
Its not limiting for safety yet, just making sure fuel commands make physical sense.

We'll turn this into a real protection limiter by adding temperature protection.
The subsystem will no longer depend only on *Wf_raw*. We will add an extra input: EGT (measured versions later). Using these, the block computes a fuel limit *Wf_Tlim* so we don't violate *EGT_max*. Then we form a pre-limited command:

Wf_pre = min(Wf_raw, Wf_Tlim)

Finally, we still apply the same actuator realism blocks (rate limit + saturation) to *Wf_pre* to produce the final *Wf_cmd*.

Wf_Tlim is computed by a small limit regulator driven by temperature error. We define:

e_T = EGT_max - EGT

$$Wf_{T\lim}(t) = Ki_T \int e_T(t)dt$$

Added proportional term

When EGT is below the limit, Wf_Tlim stays high so it does not constrain Wf_raw.
When EGT approaches or exceeds the limit, the regulator integrates this error and reduces Wf_Tlim until EGT is held at EGT_max.

Normalization

To avoid eyeballing variables, I'll classify every parameter into 4 categories.
Only a small set are true "choices"; everything else must be derived .

1) <u>Normalization</u>
Decide what 0–1 means and pick the reference baselines.

N_idle = 0.25
N_max_ref = 1
Wf_min = 0
Wf_max = 1
EGT_idle = 0.4
EGT_max = 1
T_max = 1

2) <u>Mapping</u>
Anything like $N_{eq}(Wf)$ or the EGT proxy is set using a few anchor points (idle and max conditions). Once those points are defined, slopes/intercepts (like $k_{Neq}, b_{Neq}, A_{EGT}, B_{EGT}$) come from solving the constraints, not guessing.

*N_eq = k_Neq * Wf_cmd + b_Neq*
N_ref = N_idle + throttle $*$ *(N_max_ref* $-$ *N_idle)*
Wf_idle = (N_idle - b_Neq) / k_Neq;
Wf_init = (N_init - b_Neq) / k_Neq;
*EGT = EGT_idle + A_EGT * Wf_cmd - B_EGT * N*

*EGT_init = EGT_idle + A_EGT * Wf_init - (B_EGT * N_init) ;*
*Thrust = T_max * N ^ (a_thrust)*

3) Dynamics .
Parameters like τ_N and τ_{EGT} are chosen based on response speed (first-order "how fast it moves"), not tuned randomly.

Set:
$\tau_N = 1.5\,\mathrm{s}$
$\tau_{EGT} = 3.0\,\mathrm{s}$

Simple, mid-range "generic turbofan" time constants so the spool responds faster than temperature while keeping the model stable and easy to tune.

4) Controller/limiter gains
Will be tuned or calculated from the plant dynamics and performance targets, not chosen arbitrarily.

Kp_N
Ki_N
I_N_min = 0
I_N_max = 1
Ki_T = 0.5
Kp_T = 2
dWf_up_max = 0.5
dWf_dn_max = 1

Tunning

Tuning becomes systematic by modeling the engine as a first-order system and choosing a clear time requirement (rise/settle time to the commanded speed). That requirement sets a target closed-loop time constant λ.
Use a turbofan-style rise-time requirement for λ. Take "reach 95% of commanded thrust in 5 s" as the reference target, this is following NASA's CMAPSS guidelines.

$$t_{95} \approx 3\lambda \quad \Rightarrow \quad \lambda \approx \frac{t_{95}}{3}$$

Once λ is set, the PI gains are computed from the plant gain and time constant, not guessed.

$$Kp_N = \frac{\tau_N}{k_{Neq}\lambda}, \quad Ki_N = \frac{1}{k_{Neq}\lambda}$$

Using $t_{95} = 5\,s$, the target closed-loop time constant is $\lambda \approx \frac{t_{95}}{3} \approx \frac{5}{3} = 1.67\,s$

Treat the EGT limiter like a PI controller on a first-order thermal lag, setting $\lambda_T = \tau_{EGT} = 2$ so the limiter is not faster than the thermal physics.

$$Kp_T = \frac{\tau_{EGT}}{K_{EGT}\lambda_T}, \quad Ki_T = \frac{1}{K_{EGT}\lambda_T}$$

Anti-Windup

Next step is adding real anti-windup in the Speed Governor, not just clamping integrator. When *Wf_pre* is limited by EGT protection, the PI should stop building up fuel demand; otherwise *Wf_raw* keeps growing even though the engine never receives that fuel.

A standard fix is tracking (back-calculation) anti-windup in the integrator:

$$\dot{I} = Ki_N e_N + K_{aw}(Wf_{cmd} - Wf_{raw})$$

The extra term compares what the controller *wanted* (Wf_raw) to what the engine *actually gets* (Wf_cmd). When EGT protection clamps fuel, $(Wf_{cmd} - Wf_{raw})$ is negative and it automatically "unwinds" the integrator, pulling Wf_raw back toward the limited command instead of letting it accumulate.
Use $K_{aw} = \mathrm{Ki_N}/\mathrm{Kp_N}$, where K_{aw} is the tracking anti-windup gain: it sets how quickly the integrator is forced to match the limited fuel.

$$Wf_{raw} = Kp_N e_N + \int (Ki_N e_N + K_{aw}(Wf_{cmd} - Wf_{raw}))dt + \frac{N_{ref} - b_{Neq}}{K_{Neq}}$$

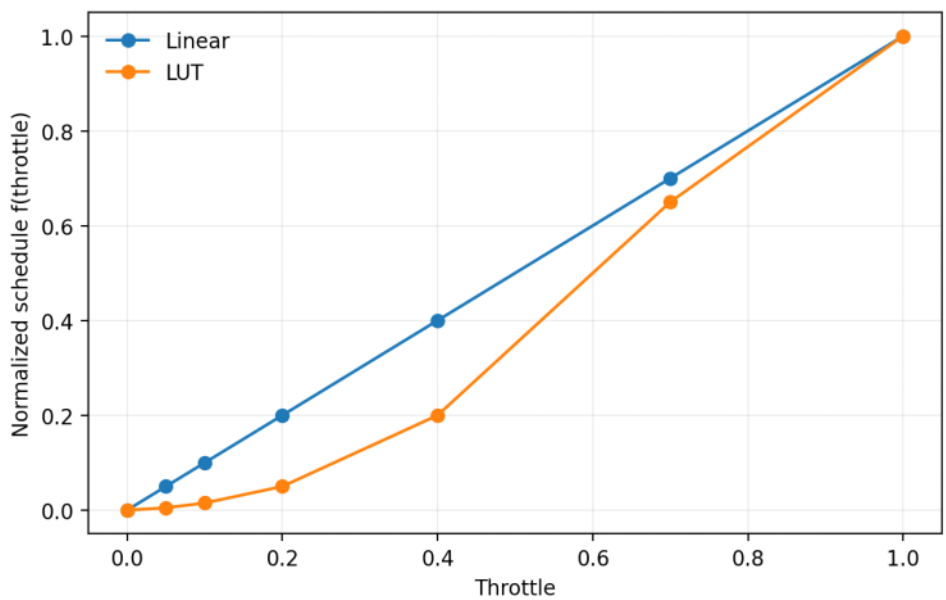
Command Mapping Schedule

The current model uses a simple linear throttle-to-speed command following eq :

N_ref = N_idle + throttle ∗ (N_maxref − N_idle)

A lookup-table schedule for N_{ref} makes the architecture more realistic by separating the user command throttle from the internally scheduled target speed.
The upgrade will replace the linear mapping with a 1-D LUT that outputs N_{ref} from throttle. The schedule will enforce $N_{ref} = N_{idle}$ at throttle $= 0$ and $N_{ref} = N_{maxref}$ at throttle $= 1$, with intermediate breakpoints shaping the response.
The rest of the controller remains unchanged

$$N_{ref} = N_{idle} + (N_{maxref} - N_{idle}) f(\text{throttle})$$



The LUT is less sensitive near idle (small throttle changes barely move N_{ref}) and becomes steeper at higher throttle so N_{ref} rises faster near the top end.

In the current simulations the difference is only mildly visible because the throttle commands spend most of the time near 0 or 1, where both the linear map and the LUT produce the same output; the effect becomes clearer with intermediate throttle ramps.

Nonlinear Spool Plant

The current spool-speed plant uses a first-order model with a linear steady-state map , which makes fuel sensitivity constant across the whole range.

$$\dot{N} = \frac{N_{eq}(Wf) - N}{\tau_N}, \quad N_{eq}(Wf) = K_{Neq}Wf + b_{Neq}$$

A more realistic behavior is diminishing sensitivity at high fuel, so the steady-state relationship is upgraded to a saturating curve while keeping the same τ_N structure.

$$N_{eq}(Wf) = b_{Neq} + (1 - b_{Neq})\frac{1 - e^{-a_{Neq}Wf}}{1 - e^{-a_{Neq}}}, \quad a_{Neq} = \frac{k_{Neq}}{1 - b_{Neq}}$$

Implementation replaces the linear N_eq computation block with the saturating expression , using a_Neq derived from existing parameters so the low-fuel slope matches the original model.
We also have to update the Fuel Feed Forward.

EGT upgrade

Right now EGT is a simple normalized proxy driven mainly by fuel with one lag ; it behaves like fuel always produces the same temperature effect.

The goal is to keep the same normalized interface but make EGT depend on operating point in a more realistic way. This will be guided by NASA C-MAPSS data trends , then implement it as a compact map/scheduled proxy inside Simulink.

For this upgrade the data needs to include fuel flow, core speed, and a temperature channel that can act as an EGT proxy. The best match is *NASA* ***N-C-MAPSS DS02***, distributed as an HDF5 (.h5) file.
T50 is a common choice for an EGT-like temperature signal.

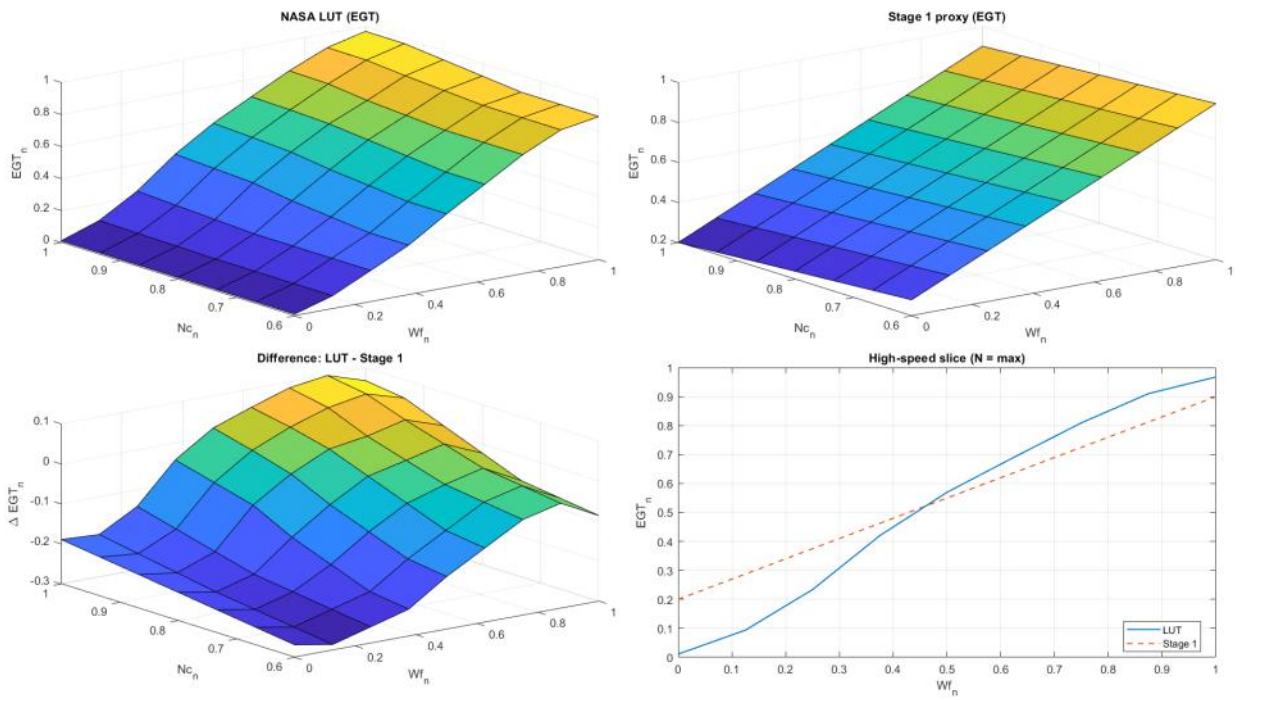
EGT = f(N , Wf)

A small data pipeline is added to build the map. The script "*build_egt_lut.m*" reads the DS02 file, extracts *Wf*, *Nc*, and *T50* from *X_s_dev* dataset, normalizes them and bins the samples on a compact grid to create a 2D LUT EGT_LUT.

$$EGT = \frac{T50 - T_{idle}}{T_{lim} - T_{idle}}$$

T_idle is taken as a low percentile of the DS02 T50 data , and T_lim is taken as a high percentile using percentiles instead of min/max to avoid outliers.

Empty bins are filled with nearest-neighbor interpolation and the table is lightly smoothed to remove blockiness. The output is saved to *data/processed/egt_lut_v1.mat*



The LUT is monotonic in fuel but curved with operating point, while Stage 1 is a constant-slope plane. The difference plot highlights where the old proxy over/underestimates EGT, and the high-speed slice shows the LUT rising more strongly at high fuel than the linear model.

The old arbitrary constants A_EGT, B_EGT, and EGT_idle are removed from the EGT path because the steady-state relationship is now data-driven by the LUT.

To force limiter activity for testing, a simple test factor is introduced: *EGT_test_gain*

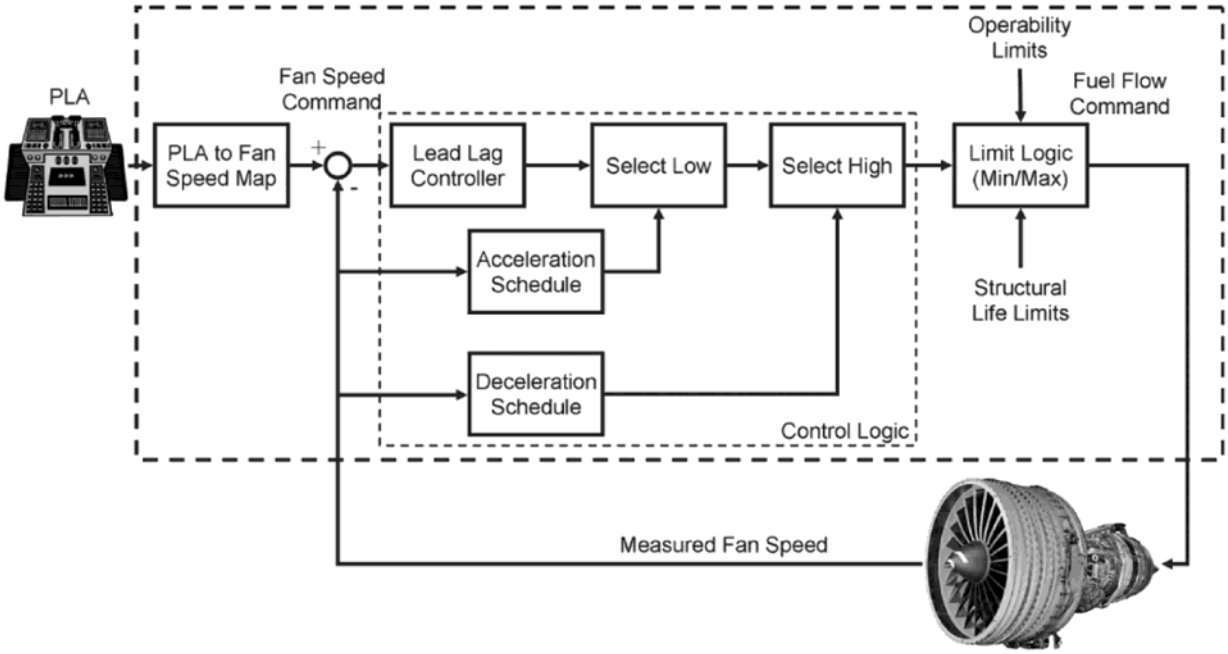


Figure 2.—Block diagram of typical modern engine control logic (image courtesy of NASA).